

NUS-ISS

*Pattern Recognition using
Machine Learning System*



Day 3 (I) - Building visual detection and recognition systems using convolutional neural networks (CNN)

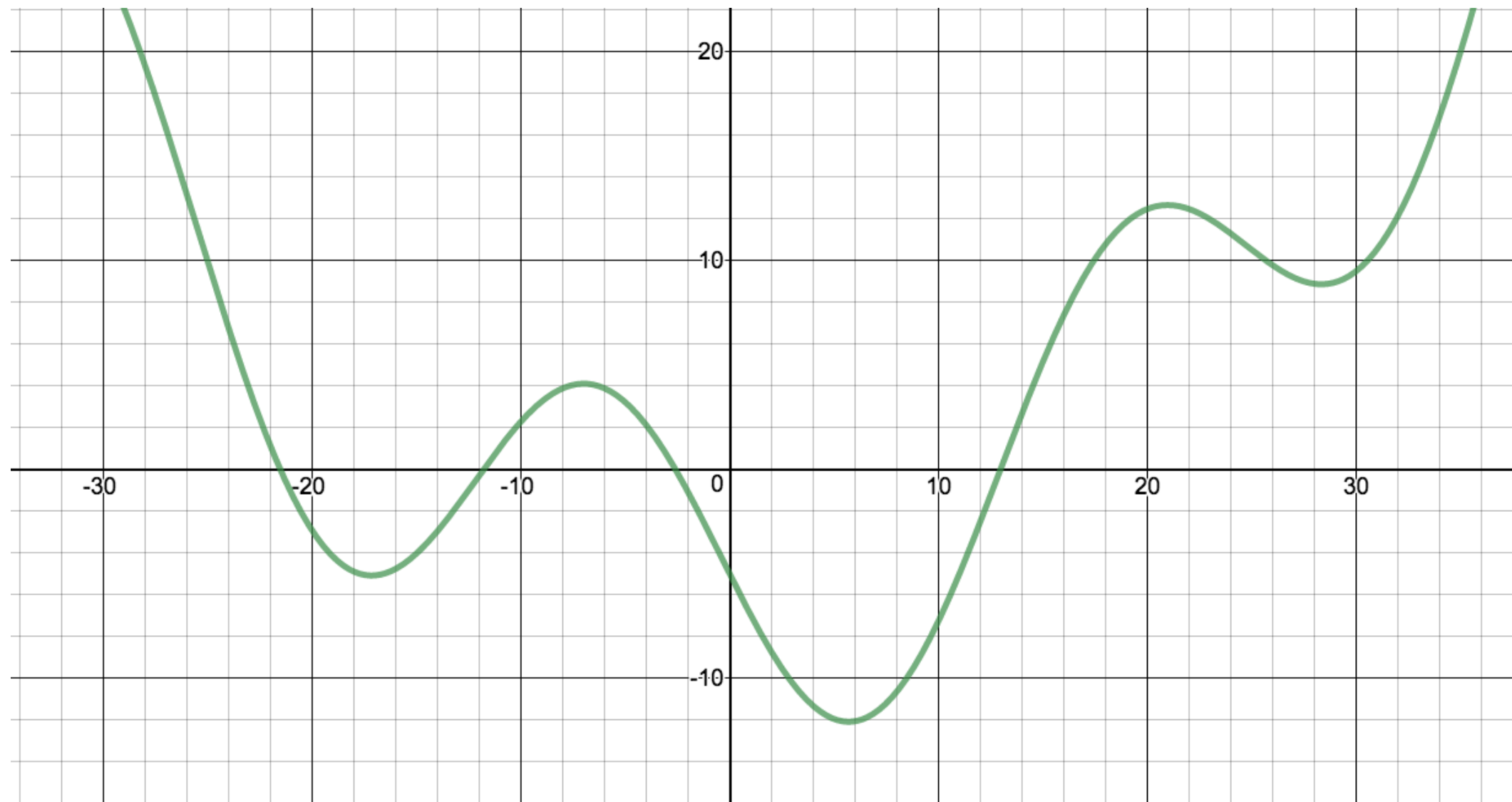
by Dr. Gary Leung

**The need of a deeper network and
functional APIs**

Representation

Can a neural network represent this complex function?

From 1D to High Dimensions: While this plot shows a 1D input (x) mapping to a 1D output (y), an image classification model does the exact same thing—just in a high-dimensional space.

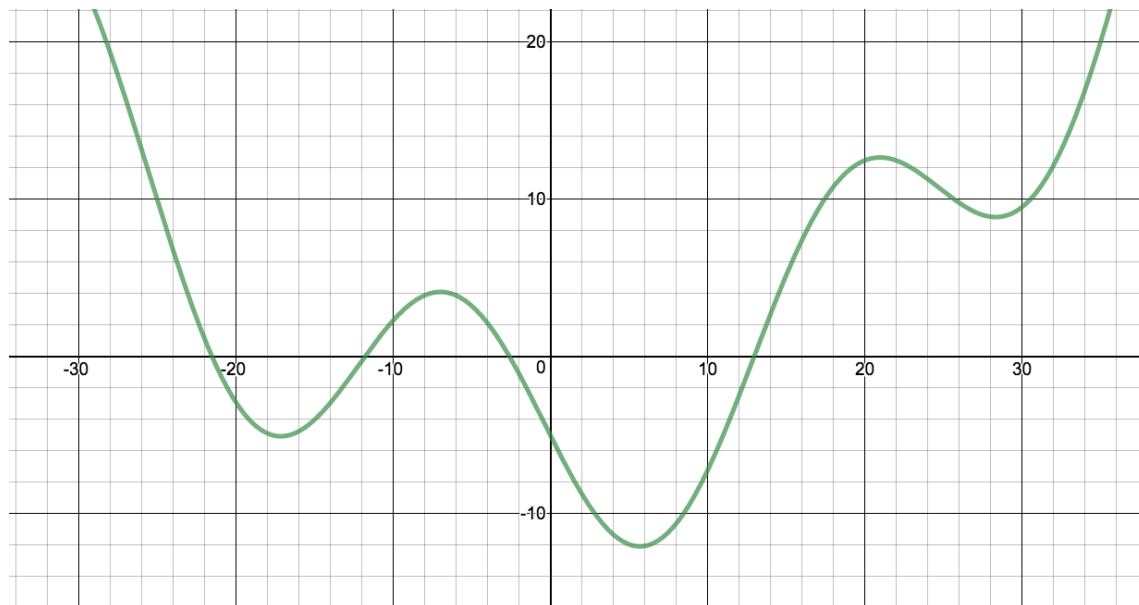


Source: By Brendan Fortuner

Universal approximation theorem

A theorem for all problems?

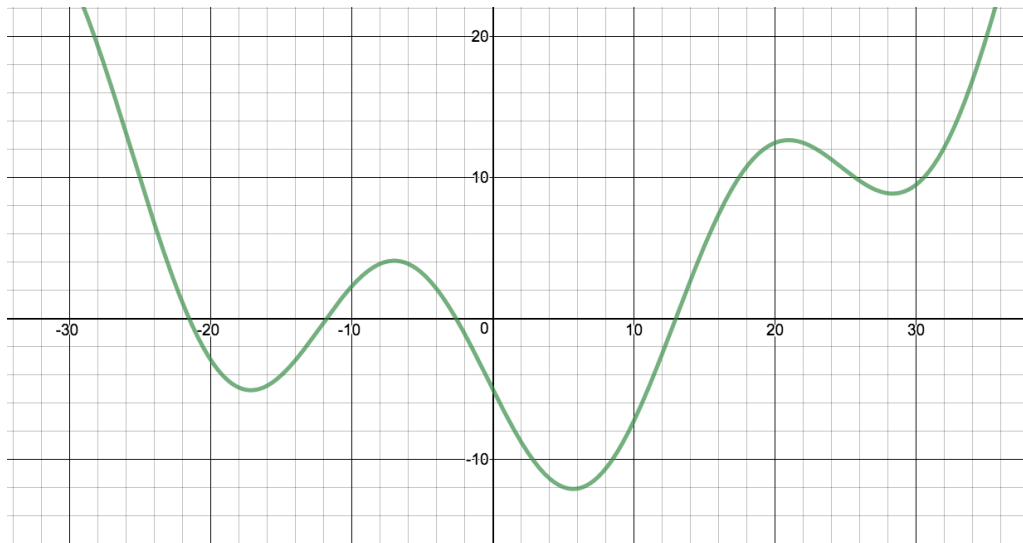
- A feedforward network with a single hidden layer is mathematically sufficient to approximate any continuous function
- What is the implication?
- If a single layer is enough, why do we need **deep neural networks**?



Source: By Brendan Fortuner

Universal approximation theorem

Things to consider



Source: By Brendan Fortuner

- What is a function? It a mapping from one set of items to another (not just mathematical equations)
- **Example:** If human can clearly discriminate various breeds of dogs, it implies a mapping function exists.
- Since this mapping function exists, **Universal Approximation Theorem** guarantees that a neural network is capable of representing it.
- In other words, a neural network is theoretically capable of mapping raw dog images to their breed classes

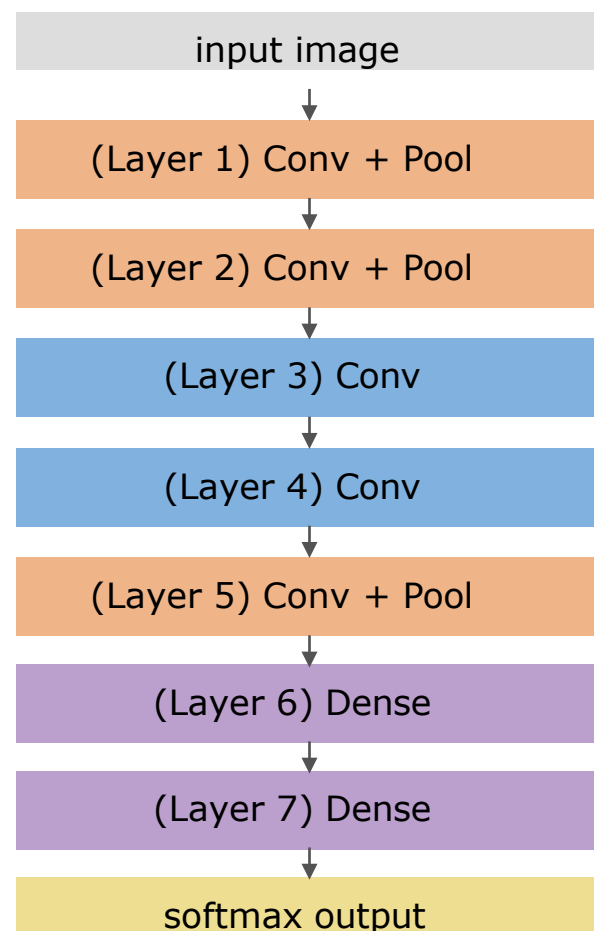
Back to the real world

The issue at hand

- Theoretical Limit:
 - **Universal Approximation Theorem** proves a single-layer network can represent any function
 - However, it provides **no practical algorithm** to actually find or train those weights
- Modern Reality:
 - Decades of empirical evidence show that network depth—not width—is the key to high performance

The importance of depth

A study on Krizhevsky *et al.* model (2012)



- AlexNet: The model that won ILSVRC 2012
- 8 layers in total, 60 million parameters, 650,000 neurons
- Re-implementation by Rob Fergus gives 18.1% top-5 error, in another words, the top-5 accuracy is 81.9%
- But what is top-5 error or top-5 accuracy?

Source: Re-implementation by Rob Fergus

Top-5 error

How about top-1 error



Persian cat

Source: <http://www.vetstreet.com/cats/>

Top-1 error

Chihuahua (0.4)
Hyena (0.25)
Koala (0.15)
Persian cat (0.1)
Burmese cat (0.02)

Considered **incorrect**

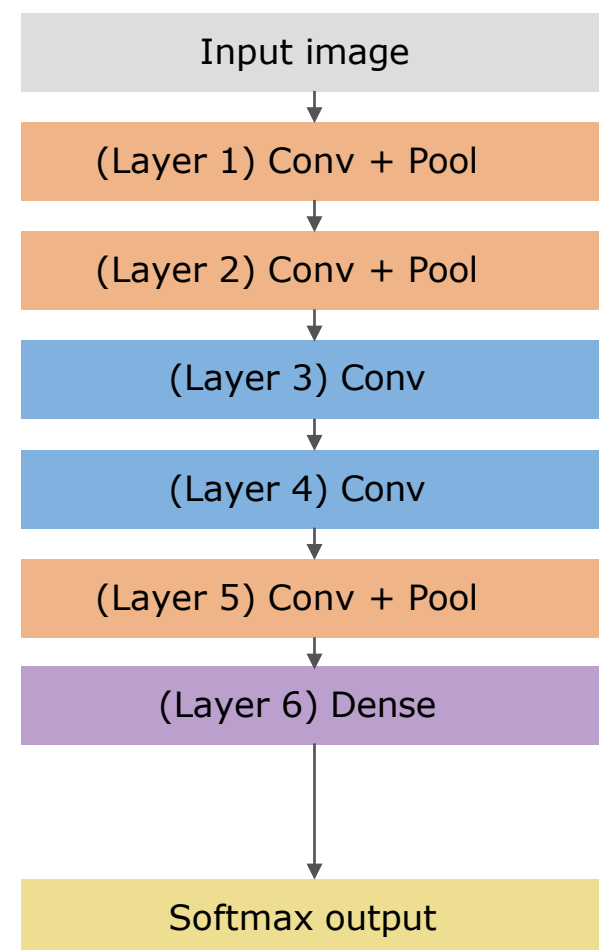
Top-5 error

Chihuahua (0.4)
Hyena (0.25)
Koala (0.15)
Persian cat (0.1)
Burmese cat (0.02)

Considered **correct**

The importance of depth

A study on Krizhevsky *et al.* model (2012)

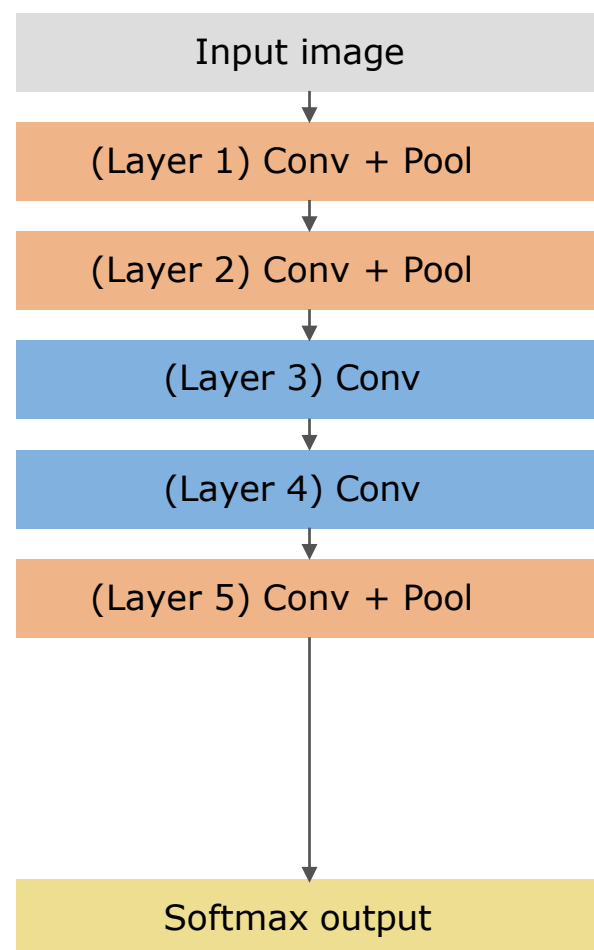


- What would happen if we remove layer 7, the fully connected layers?
- When layer 7 is removed, the model will have 16 million less parameters, from 60 million to 44 million
- How about the performance? There is only 1.1% drop in performance! From 81.9% to 80.8% for top-5 accuracy

Source: Re-implementation by Rob Fergus

The importance of depth

A study on Krizhevsky *et al.* model (2012)

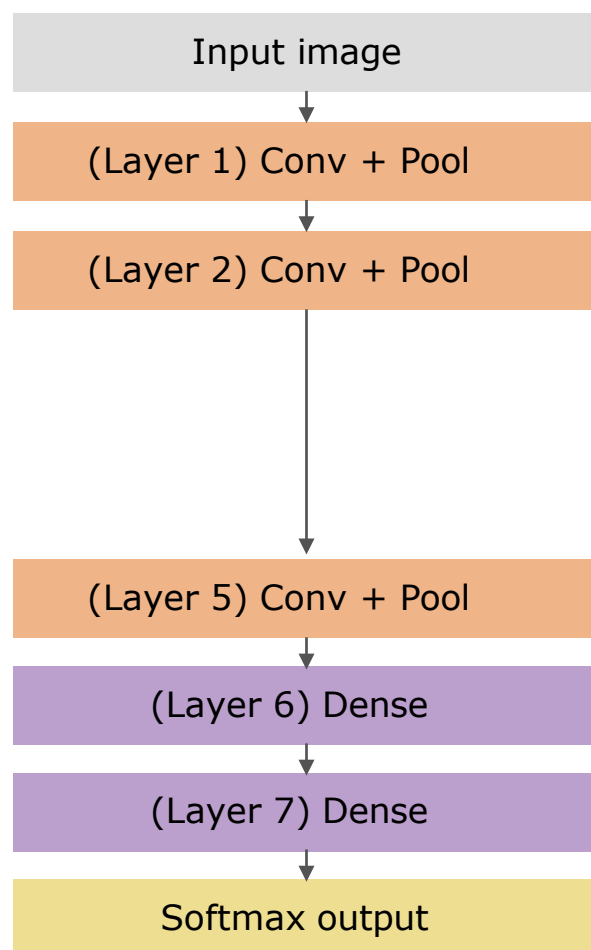


- How about we remove both layer 6 and layer 7, the two fully connected layers?
- The model will have 50 million less parameters compared to the original model, from 60 million to 10 million
- 83% of the initial number of parameters eliminated
- How about the performance? Only 5.7% drop in performance, from 81.9% to 76.2%
- Note: without the classifier the model still works fine

Source: Re-implementation by Rob Fergus

The importance of depth

A study on Krizhevsky *et al.* model (2012)

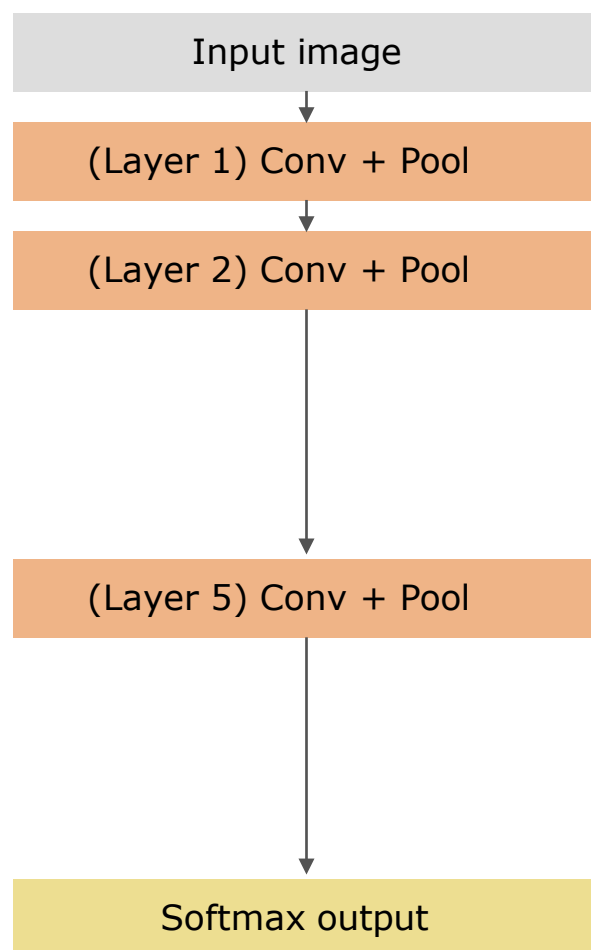


Source: Re-implementation by Rob Fergus

- Now let's try to remove upper layers, the feature extractor. Remove layer 3 and 4
- The model has 1 million less parameters compared to the original model, from 60 million to 59 million
- But there is a 3.0% drop in performance, from 81.9% to 78.9%
- Reminder: When we remove layer 7, the model loses 16 million parameters, but the drop in performance is only 1.1%
- Observation:
 - 1) A large number of parameters does not necessarily make a network powerful
 - 2) Removing feature extractor layers hurts a network more

The importance of depth

A study on Krizhevsky *et al.* model (2012)

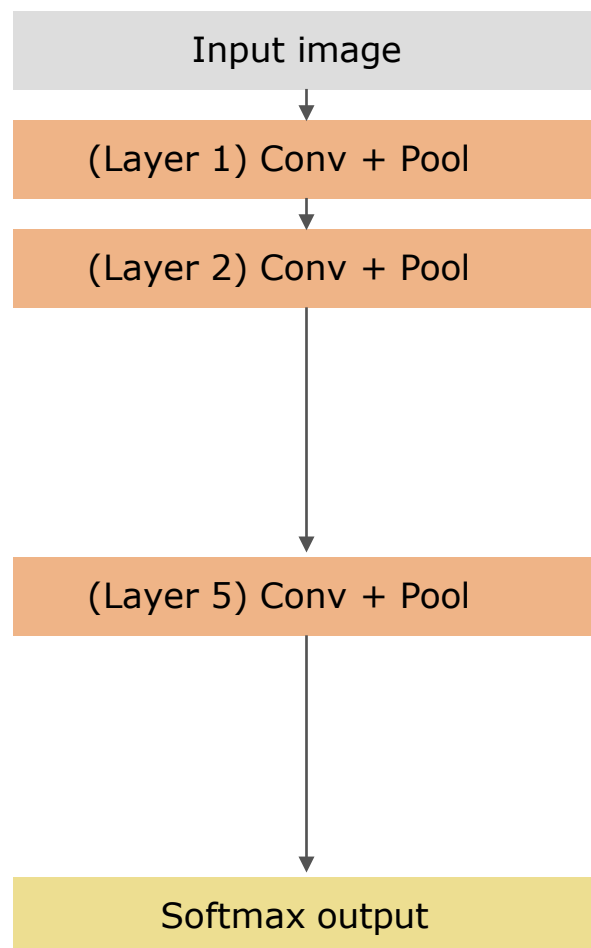


Source: Re-implementation by Rob Fergus

- Remove convolutional layers 3 and 4 together with fully connected layers 6 and 7
- After removal, the network has only 4 layers left. The model has 9 million parameters
- Reminder: When we remove layers **3** and **4**, the model has **59** million parameters, and performance drops by **3.0%**
- Reminder: When we remove layers **6** and **7**, the model has **10** million parameters, and performance drops by **5.7%**
- Now with layers **3**, **4**, **6** and **7** removed, the model has **9** million parameters. What should the performance drop be?

The importance of depth

A study on Krizhevsky *et al.* model (2012)



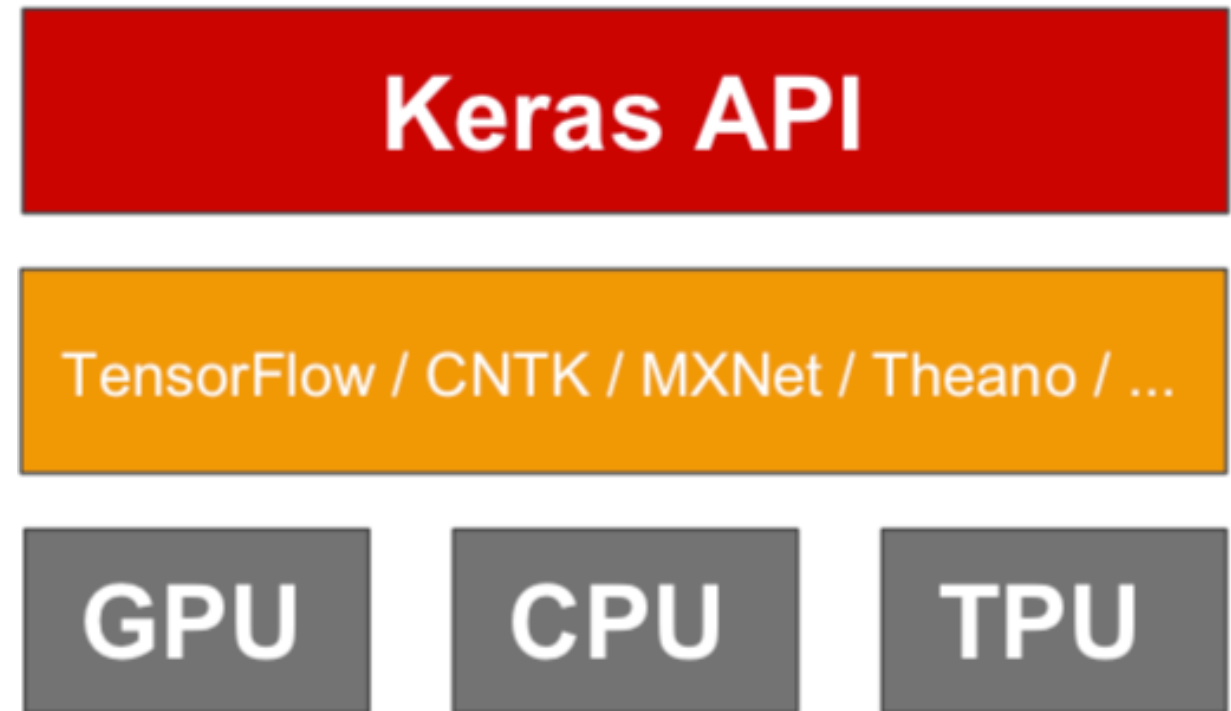
Source: Re-implementation by Rob Fergus

- With layer **3**, **4**, **6** and **7** removed, the model has **9** million parameters. What should the performance drop be?
- Performance drops substantially: **33.5%**. Top-5 accuracy falls from **81.9%** to **48.4%**
- In previous experiments, the model still had at least **6** layers. Regardless of which layers were removed, the network retained reasonably good accuracy, above **75%**
- But when number of layers is reduced by half, performance suffers greatly. The power of a deep network comes from its depth
- **Depth is key**

Keras

What is Keras

The basic architecture

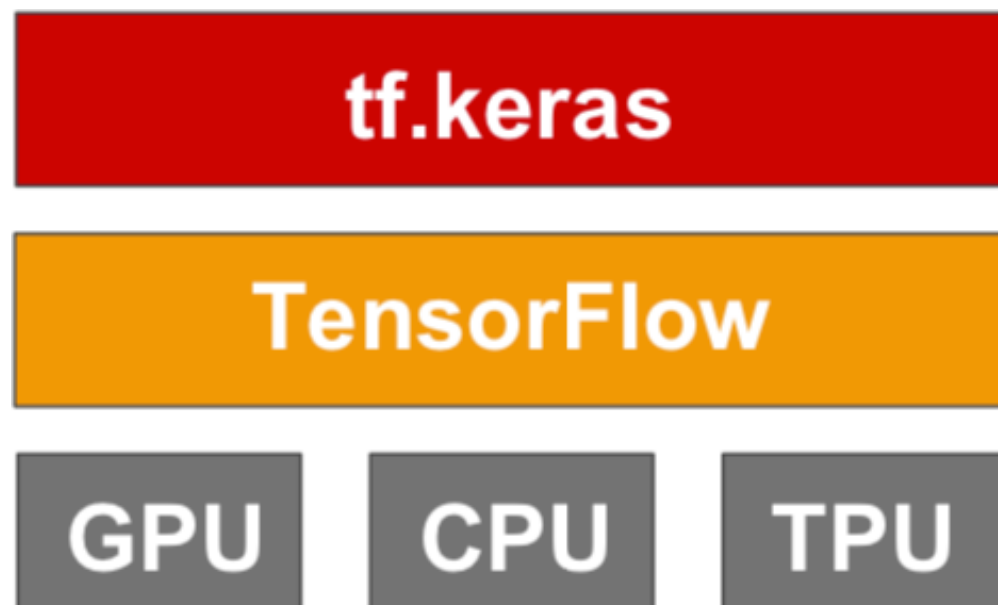


Source: Francois Chollet

Keras + TensorFlow

The official high-level API of TensorFlow

- Possible to build deep net solely using Tensorflow, but too many repetition
- Need to define every detail (weight, bias, initialization and etc.)
- Since TensorFlow v1.4, Keras is part of the core
- In TensorFlow v1.13, we have tensorflow.keras module
- From TensorFlow v2.0 onward, Keras API became **the preferred way** to build neural networks

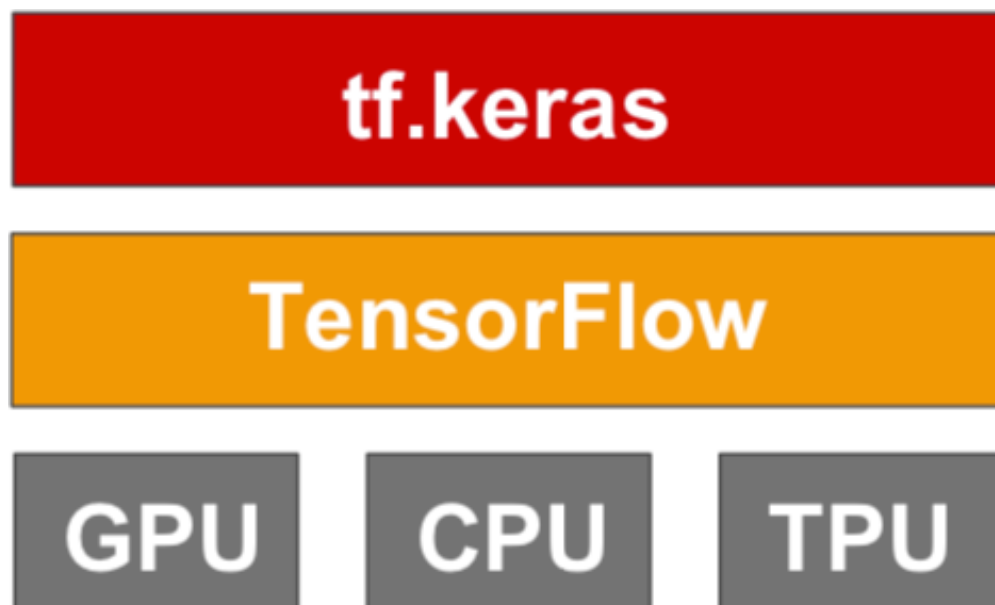


Source: Francois Chollet

Keras + TensorFlow

The official high-level API of TensorFlow

- In tensorflow.keras, we have full Keras API
- Better optimized for TensorFlow
- Better integration with TF-specific features
- Estimator API, Eager execution, tf.data



Source: Francois Chollet

Keras deployment

Broad range of options for deploying models

- Server: TensorFlow Serving
- Browser: TensorFlow.js
- Mobile: TensorFlow Lite, Core ML
- Edge: Raspberry Pi
- Java / JVM: TensorFlow Java or exported model formats

Source: Francois Chollet

Keras

Three API styles

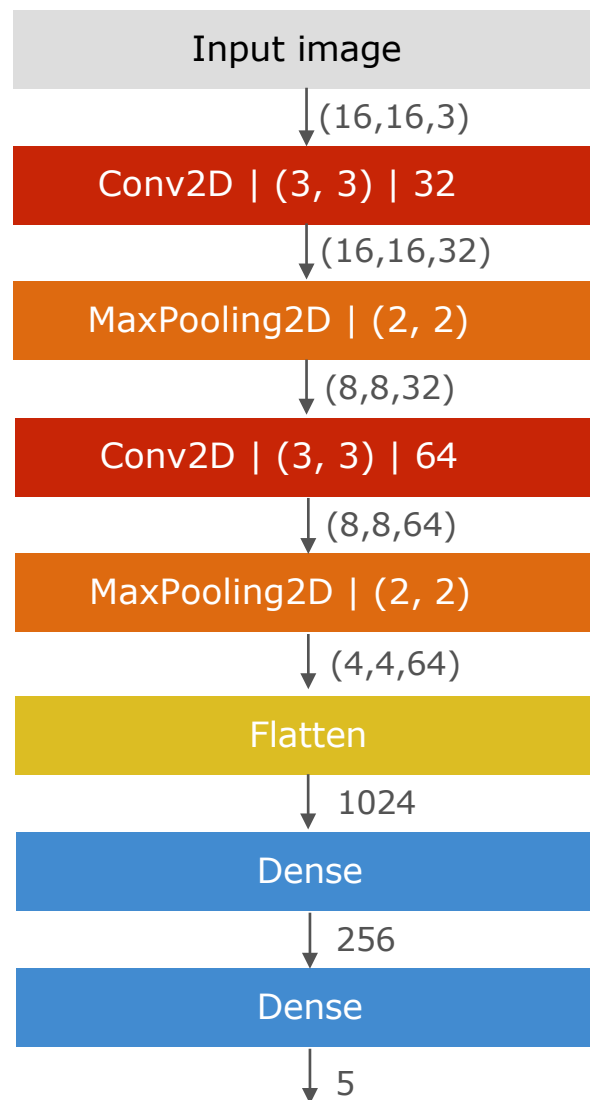
- Sequential model
 - Simple
 - Best for single-input, single-output layer stacks
 - Suitable for 70%+ of use cases
- Functional API
 - Works like connecting Lego blocks
 - Supports multi-input, multi-output, shared layers, and arbitrary static graph topologies
 - Suitable for 95%+ of use cases
- Model subclassing
 - Maximum flexibility
 - Larger potential error surface

Source: Francois Chollet

Keras

Sequential model

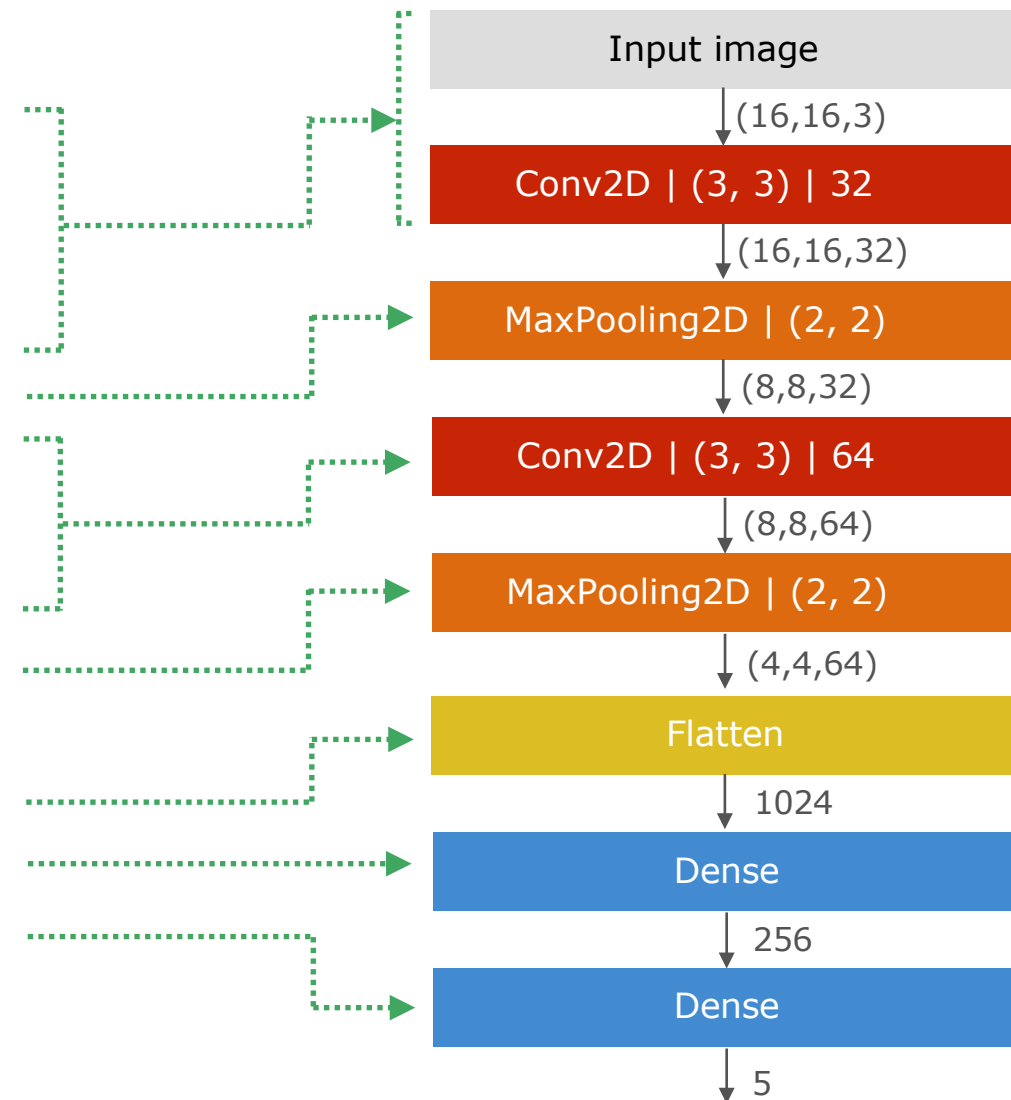
- Adding layers one by one
- `input_shape` must be present in the first layer



Keras

Sequential model example

```
> def createSeqModel():  
    model = Sequential()  
    model.add(Conv2D(32,(3,3),  
                    input_shape=(16,16,3),  
                    padding='same',  
                    activation='relu'))  
    model.add(MaxPooling2D(pool_size=(2,2)))  
    model.add(Conv2D(64,(3,3),  
                    padding='same',  
                    activation='relu'))  
    model.add(MaxPooling2D(pool_size=(2,2)))  
  
    model.add(Flatten())  
    model.add(Dense(256,activation='relu'))  
    model.add(Dense(5,activation='softmax'))  
  
    model.compile(loss='categorical_crossentropy',  
                  optimizer='rmsprop',  
                  metrics=['accuracy'])  
  
    return model
```



Keras

Sequential model

- Create the model and show the model summary
 - > modelSeq = createSeqModel()
 - > modelSeq.summary()

Layer (type)	Output Shape	Param #
=====		
conv2d (Conv2D)	(None, 16, 16, 32)	896

max_pooling2d (MaxPooling2D)	(None, 8, 8, 32)	0

conv2d_1 (Conv2D)	(None, 8, 8, 64)	18496

max_pooling2d_1 (MaxPooling2D)	(None, 4, 4, 64)	0

flatten (Flatten)	(None, 1024)	0

dense (Dense)	(None, 256)	262400

dense_1 (Dense)	(None, 5)	1285
=====		
Total params: 283,077		
Trainable params: 283,077		
Non-trainable params: 0		

Keras

Sequential model

- The problem

- Not suitable for multiple inputs
- Not suitable for multiple outputs
- Tensors flow in a single direction
- No branching or merging
- Difficult to reuse/shared layers

```
> def createSeqModel():
    model = Sequential()
    model.add(Conv2D(32,(3,3),
                    input_shape=(16,16,3),
                    padding='same',
                    activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(64,(3,3),
                    padding='same',
                    activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))

    model.add(Flatten())
    model.add(Dense(256,activation='relu'))
    model.add(Dense(5,activation='softmax'))

    model.compile(loss='categorical_crossentropy',
                  optimizer='rmsprop',
                  metrics=['accuracy'])

    return model
```

Multiple inputs?

An video and a question

- Why do we need multiple inputs?
- Example: Video question answering
- We want the model to analyse a video and answer a question about it
- Instead of analysing the video alone, we also provide a question, such as:
“Who is the person?”
“What is the person doing?”



Source: people.com

Multiple inputs?

An video and a question

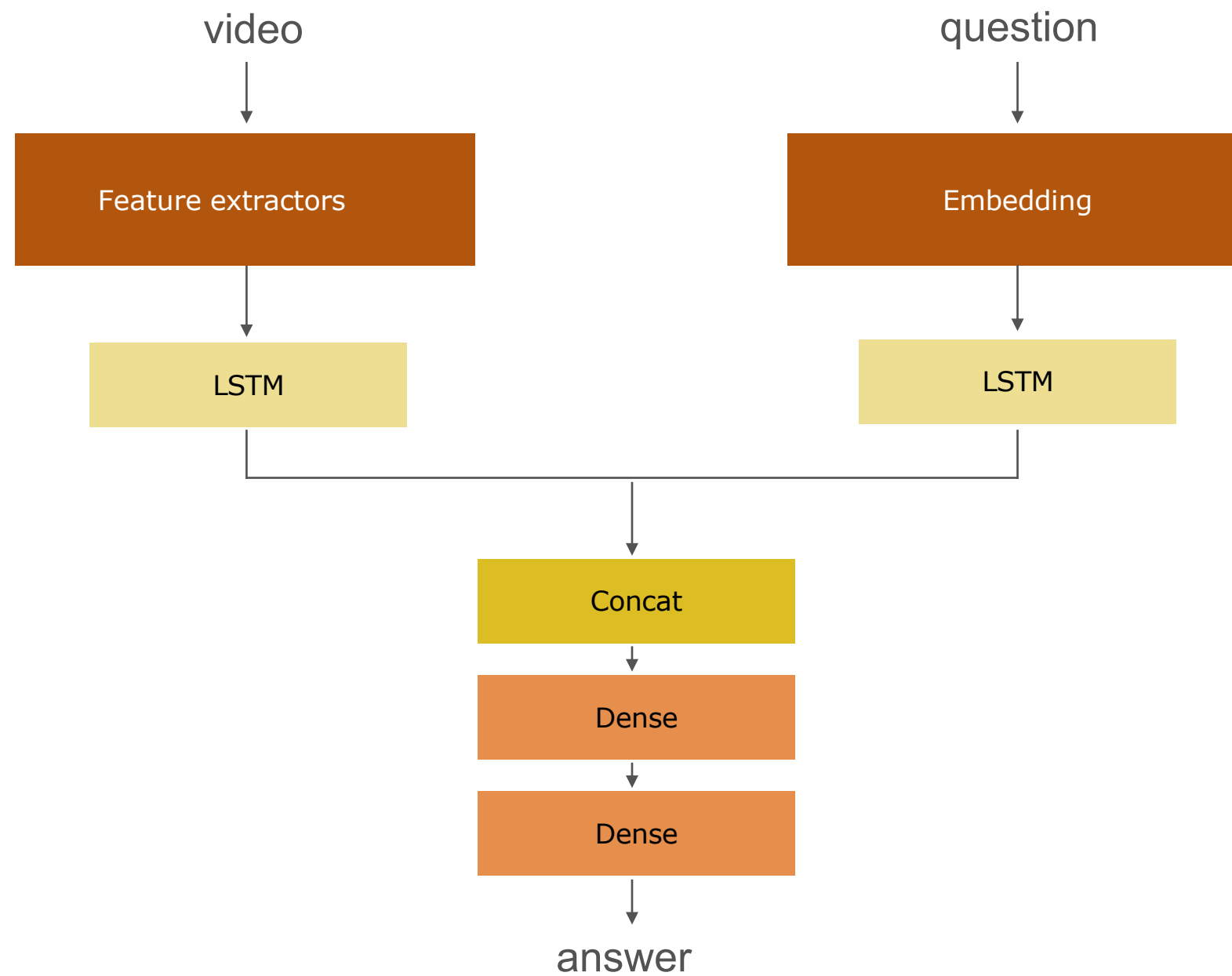
- "What is the person doing?"
"Tidying"
- "Who is the person?"
"Kondo"



Source: people.com

Multiple inputs

A video and a question

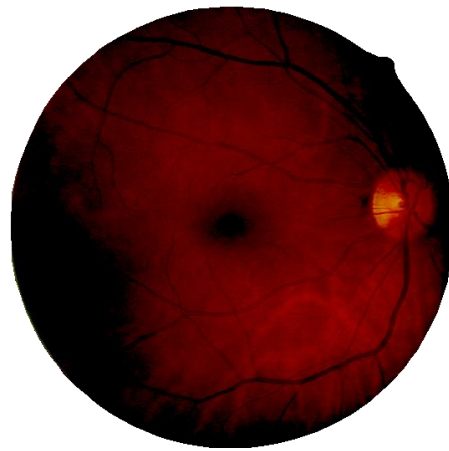


Multiple outputs?

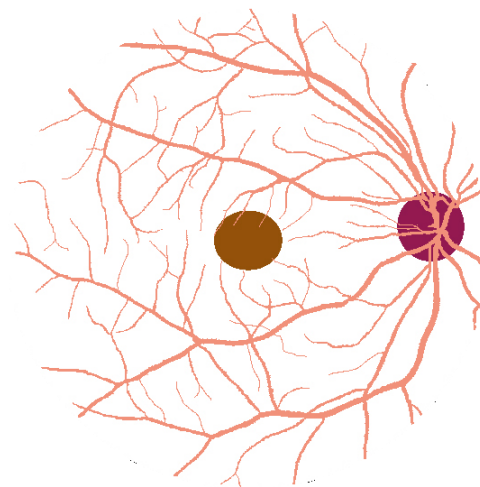
An enhanced image and
a segmentation map

- Why do we need multiple outputs?
- Example: Retinal image analysis
- A single model can produce:
 - An adjusted/enhanced retinal image
 - A segmentation map of anatomical features
- Instead of training two separate models, one can learn both tasks together

input image



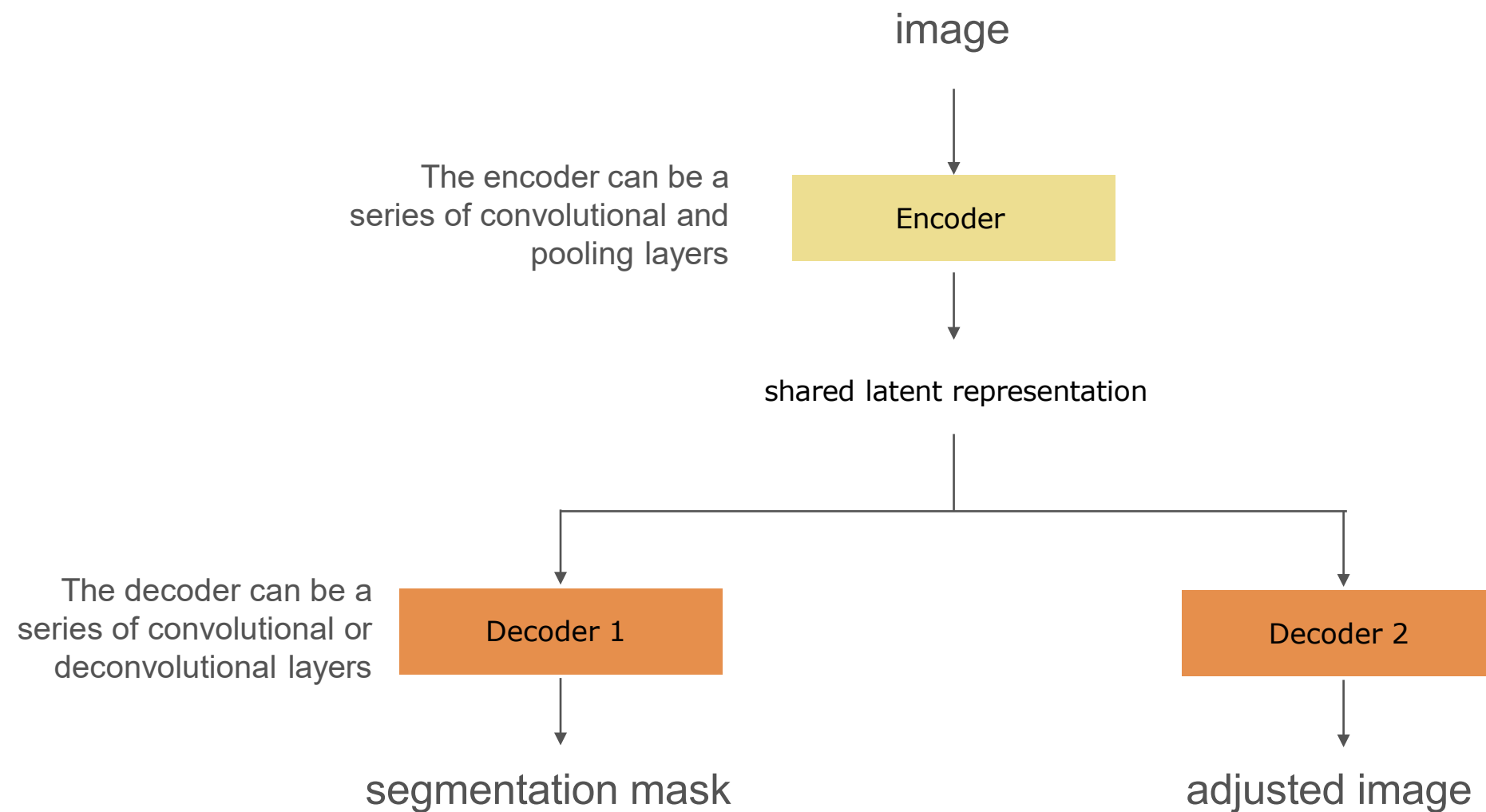
enhanced retinal
image



retinal structure
segmentation

Multiple outputs

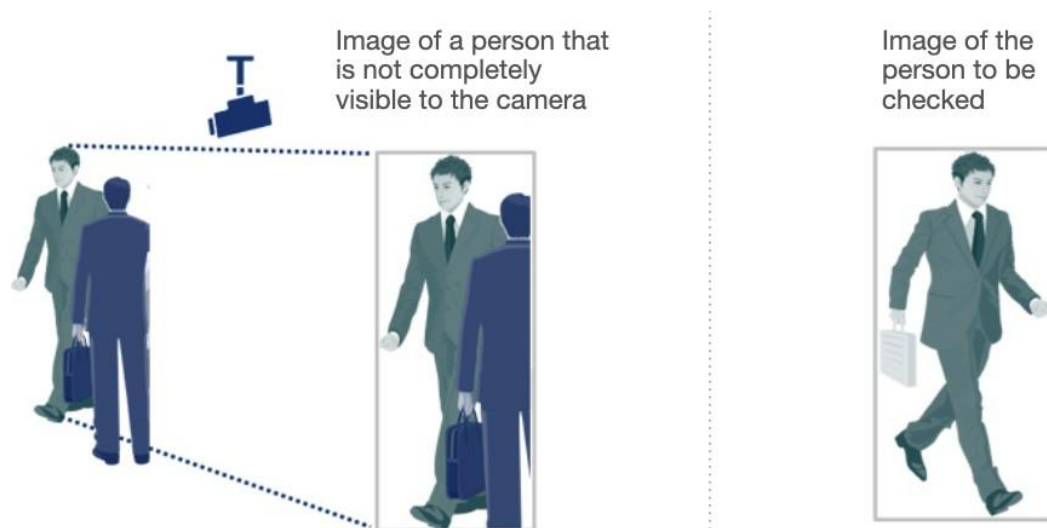
An adjusted image and a segmentation mask



Why shared layers?

Person re-identification

- Goal: identify whether an employee has come to work
- Input 1: image captured by the office entrance camera
- Input 2: reference image of the employee from the database
- This is challenging because the person may be partially blocked, or seen from a different angle

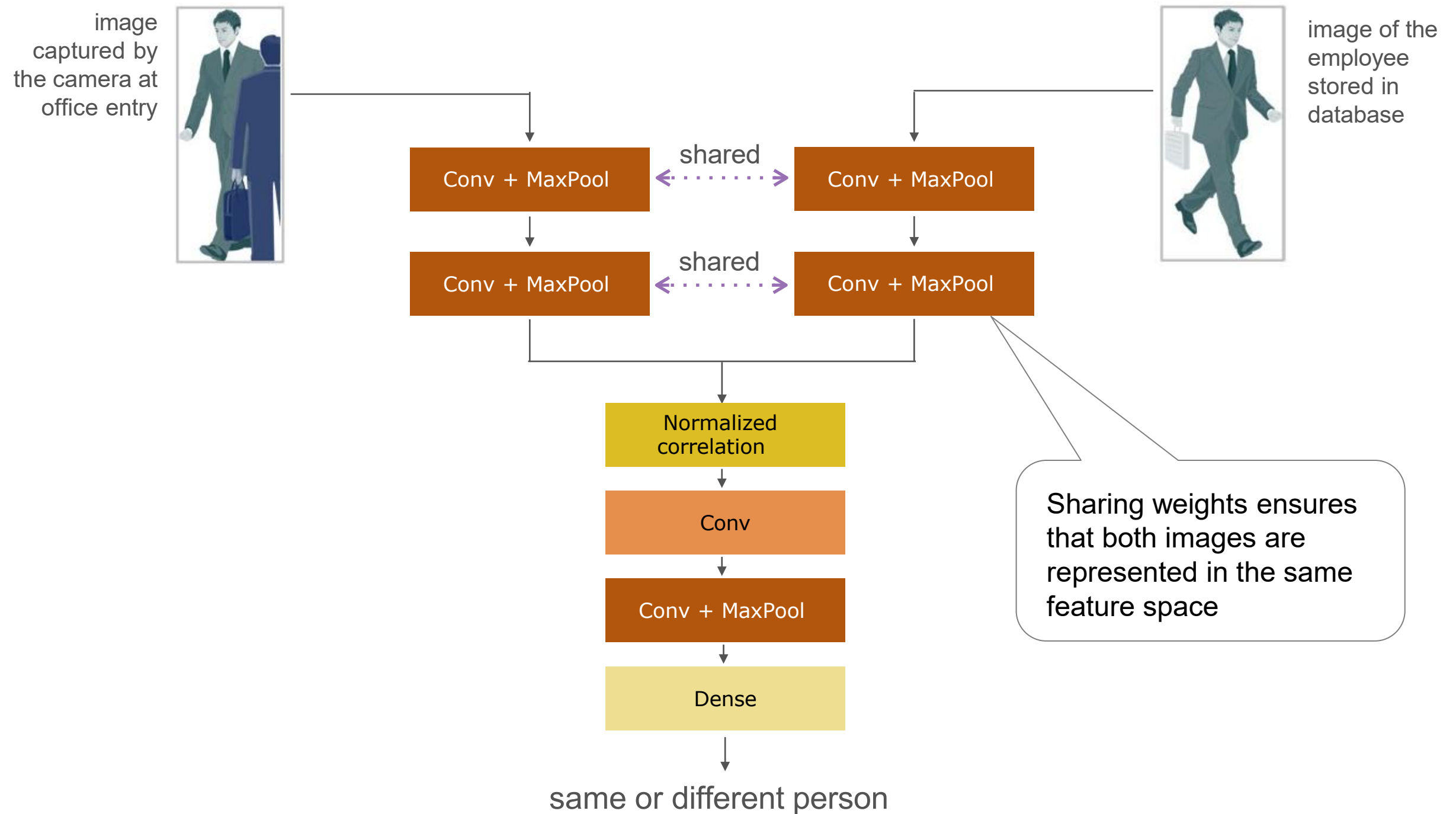


Source: <https://yellrobot.com/nec-person-reidentification-technology-can-identify-body-shape/>

Shared layers

Person re-identification

- The model compares two images
- Both images are processed by the same feature extractor



Keras

Functional API approach

- First, create the individual layers
- Second, chain or connect the layers in the way we desire the flow of tensor to be
- Third, put the input(s) and the output(s) into the `Model()` function to establish the entire network

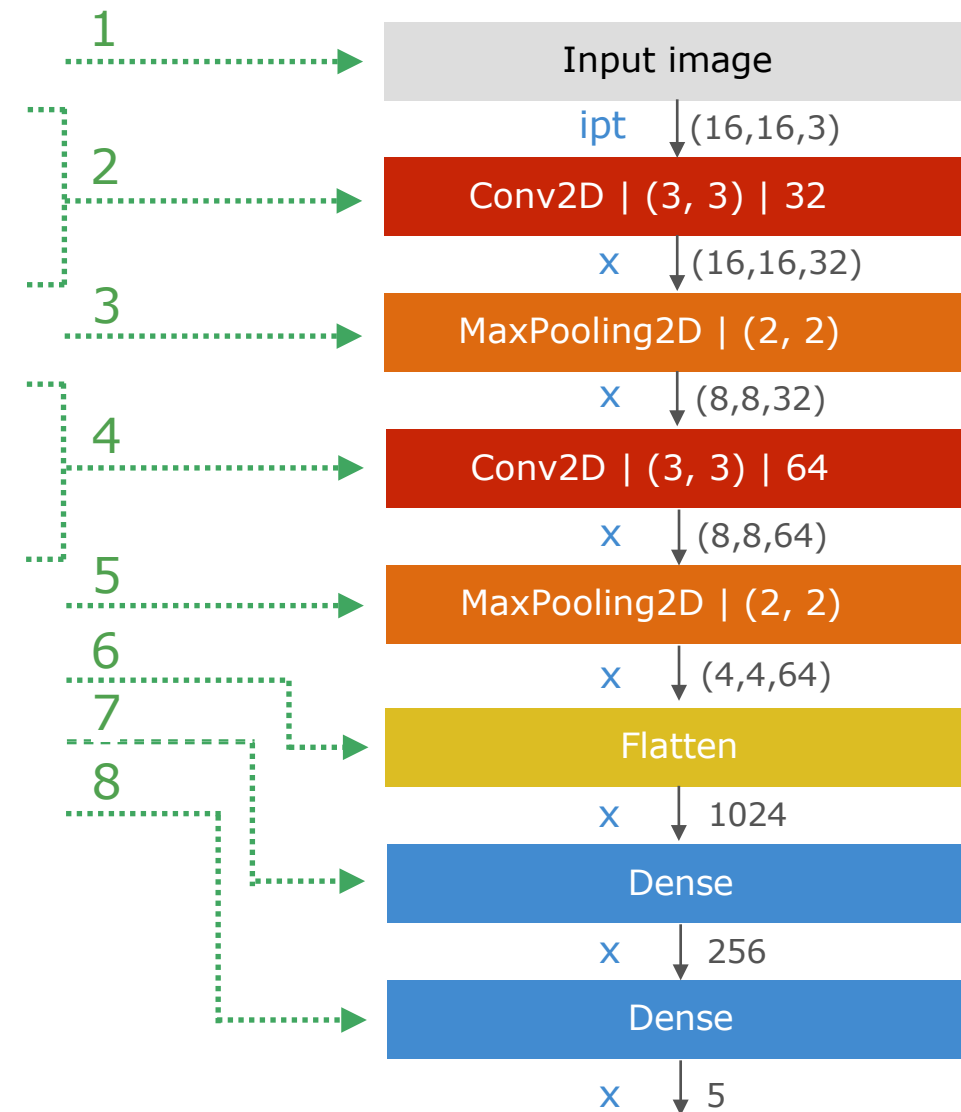


Source: <https://pixabay.com/illustrations/gear-gears-function-interaction-240137/>

Keras

Functional APIs

```
> def createFuncModel():  
    input      = Input(shape=(16,16,3))  
    x          = Conv2D(32,(3,3),  
                        padding='same',  
                        activation='relu')(ipt)  
    x          = MaxPooling2D(pool_size=(2,2))(x)  
    x          = Conv2D(64,(3,3),  
                        padding='same',  
                        activation='relu')(x)  
    x          = MaxPooling2D(pool_size=(2,2))(x)  
    x          = Flatten()(x)  
    x          = Dense(256,activation='relu')(x)  
    x          = Dense(5,activation='softmax')(x)  
  
    model      = Model(inputs=input,outputs=x)  
  
    model.compile(loss='categorical_crossentropy',  
                  optimizer='rmsprop',  
                  metrics=['accuracy'])  
  
    return model
```



Keras

Comparison

Functional API

```
> def createFuncModel():
    ipt      = Input(shape=(16,16,3))
    x        = Conv2D(32,(3,3),
                      padding='same',
                      activation='relu')(ipt)
    x        = MaxPooling2D(pool_size=(2,2))(x)
    x        = Conv2D(64,(3,3),
                      padding='same',
                      activation='relu')(x)
    x        = MaxPooling2D(pool_size=(2,2))(x)
    x        = Flatten()(x)
    x        = Dense(256,activation='relu')(x)
    x        = Dense(5,activation='softmax')(x)

    model    = Model(inputs=ipt,outputs=x)

    model.compile(loss='categorical_crossentropy',
                  optimizer='rmsprop',
                  metrics=['accuracy'])

    return model
```

Sequential model

```
> def createSeqModel():
    model    = Sequential()
    model.add(Conv2D(32,(3,3),
                     input_shape=(16,16,3),
                     padding='same',
                     activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))
    model.add(Conv2D(64,(3,3),
                     padding='same',
                     activation='relu'))
    model.add(MaxPooling2D(pool_size=(2,2)))

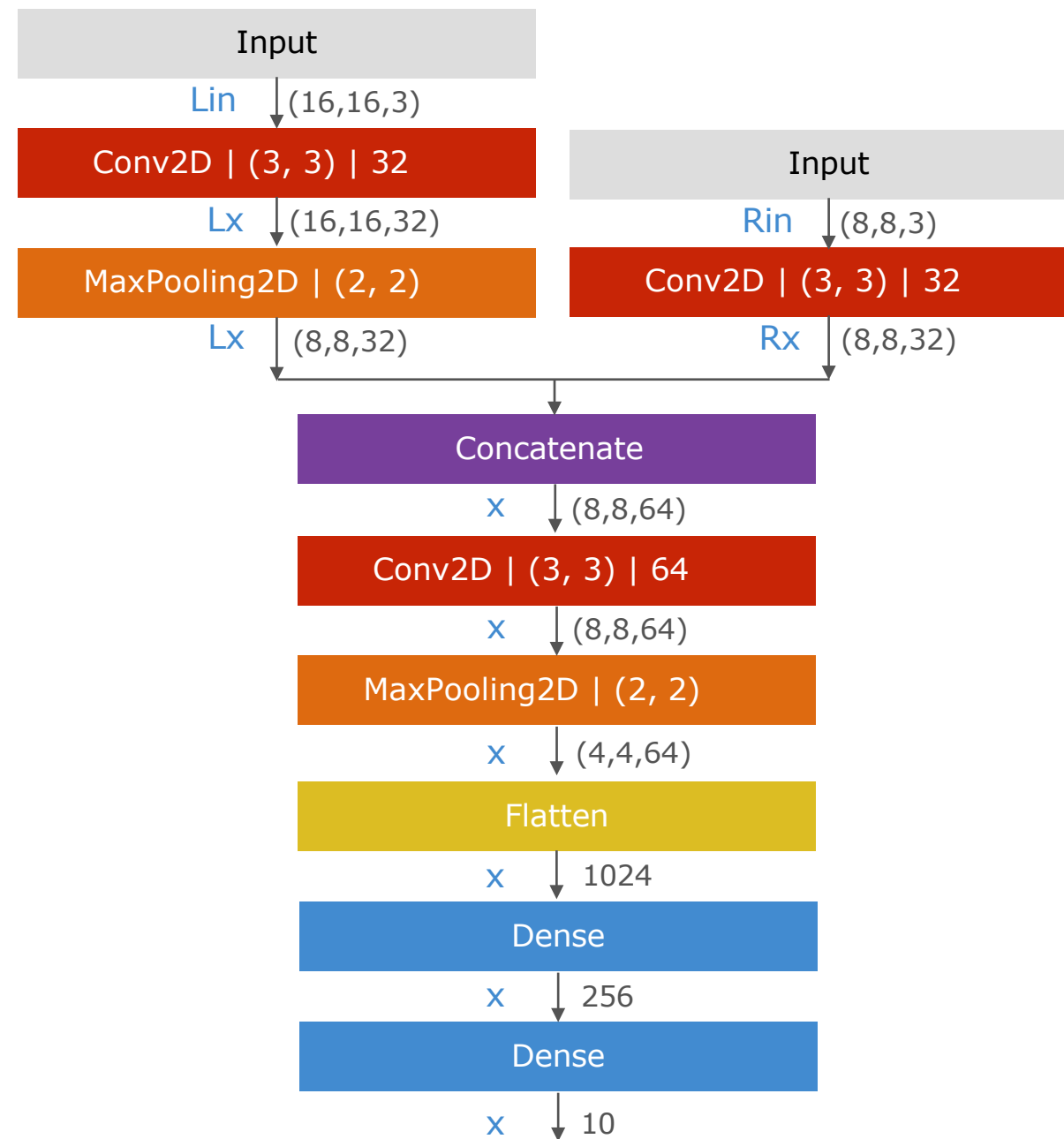
    model.add(Flatten())
    model.add(Dense(256,activation='relu'))
    model.add(Dense(5,activation='softmax'))

    model.compile(loss='categorical_crossentropy',
                  optimizer='rmsprop',
                  metrics=['accuracy'])

    return model
```

'Y' shape architecture

how to create?



'Y' shape architecture

the code

```
> def createDualInputModel():
```

```
    Lin    = Input(shape=(16,16,3))
```

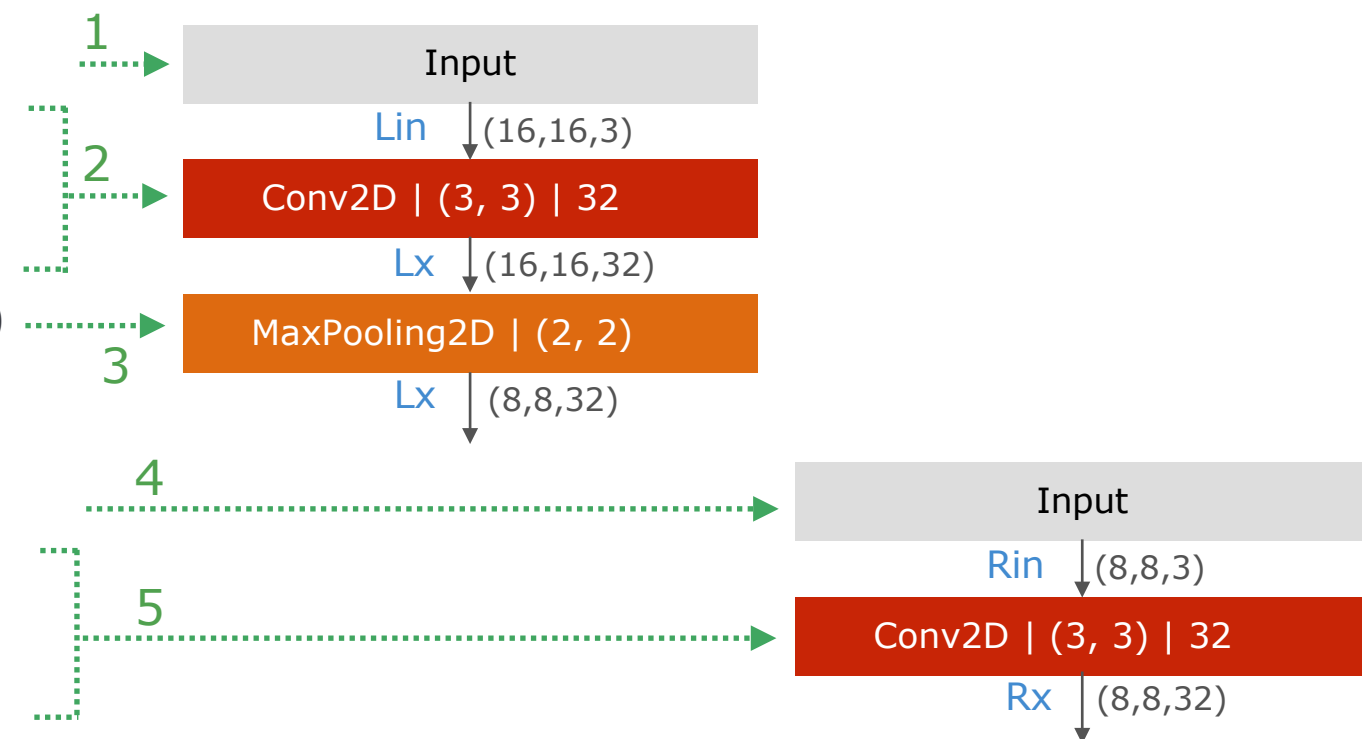
```
    Lx     = Conv2D(32,(3,3),  
                  padding='same',  
                  activation='relu')(Lin)
```

```
    Lx     = MaxPooling2D(pool_size=(2,2))(Lx)
```

```
    Rin    = Input(shape=(8,8,3))
```

```
    Rx     = Conv2D(32,(3,3),  
                  padding='same',  
                  activation='relu')(Rin)
```

```
    ...
```



'Y' shape architecture

the code

```
> def createDualInputModel():
```

```
...
```

```
x = concatenate([Lx,Rx],axis=-1)
```

```
x = Conv2D(64,(3,3),  
          padding='same',  
          activation='relu')(x)
```

```
x = MaxPooling2D(pool_size=(2,2))(x)
```

```
x = Flatten()(x)
```

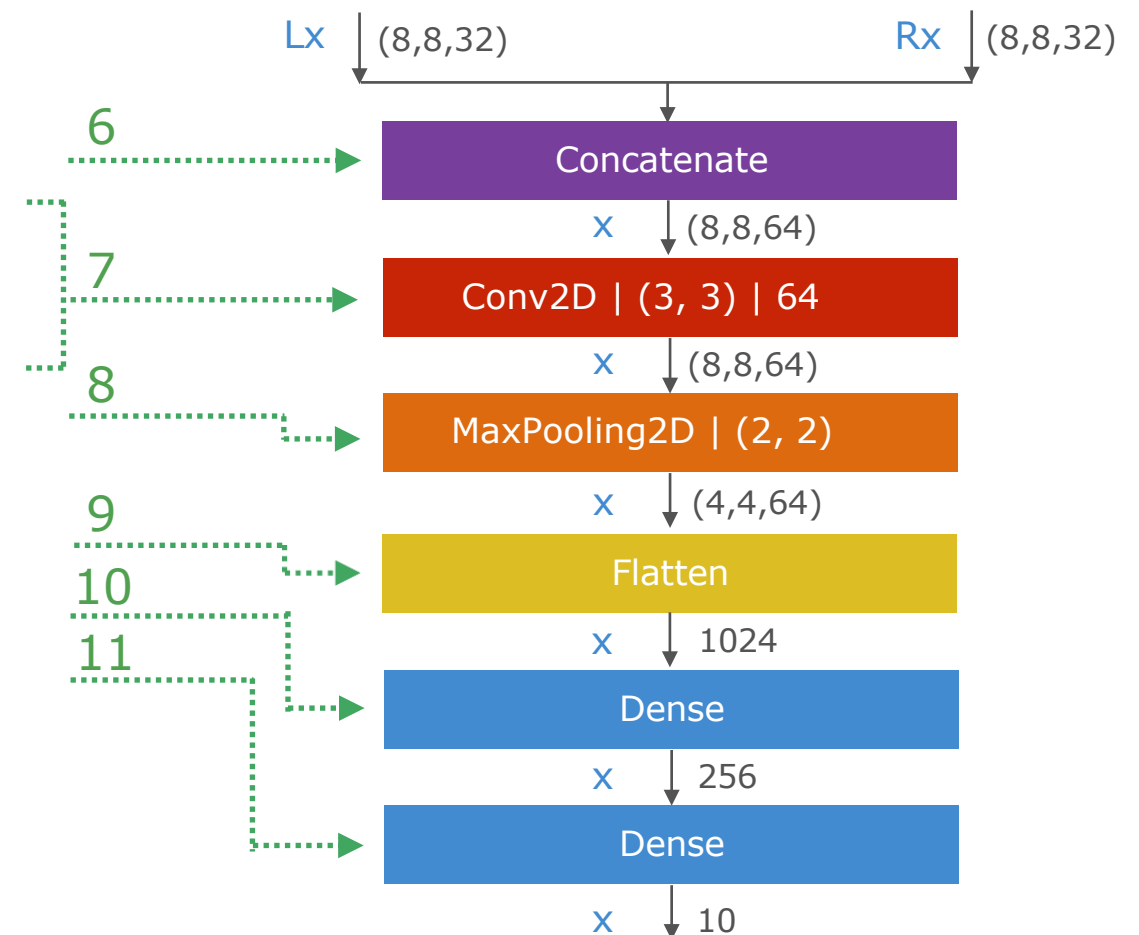
```
x = Dense(256,activation='relu')(x)
```

```
x = Dense(10,activation='softmax')(x)
```

```
model = Model(inputs=[Lin,Rin],outputs=x)
```

```
model.compile(loss='categorical_crossentropy',  
              optimizer='rmsprop',  
              metrics=['accuracy'])
```

```
return model
```



'Y' shape architecture

Putting all together

- Remember to include the below before the function

```
from tensorflow.keras.layers import concatenate
```

```
> def createDualInputModel():
```

```
    Lin      = Input(shape=(16,16,3))
    Lx       = Conv2D(32,(3,3),padding='same',activation='relu')(Lin)
    Lx       = MaxPooling2D(pool_size=(2,2))(Lx)

    Rin      = Input(shape=(8,8,3))
    Rx       = Conv2D(32,(3,3),padding='same',activation='relu')(Rin)

    x        = concatenate([Lx,Rx],axis=-1)
    x        = Conv2D(64,(3,3),padding='same',activation='relu')(x)
    x        = MaxPooling2D(pool_size=(2,2))(x)

    x        = Flatten()(x)
    x        = Dense(256,activation='relu')(x)
    x        = Dense(10,activation='softmax')(x)

    model    = Model(inputs=[Lin,Rin],outputs=x)
    model.compile(loss='categorical_crossentropy',
                  optimizer='rmsprop',
                  metrics=['accuracy'])

    return model
```

'Y' shape architecture

Inspecting model summary

- Create the model and show the model summary

```
> modelDual = createDualInputModel()  
> modelDual.summary()
```

Layer (type)	Output Shape	Param #	Connected to
=====			
input_1 (InputLayer)	(None, 16, 16, 3)	0	
conv2d (Conv2D)	(None, 16, 16, 32)	896	input_1[0][0]
input_2 (InputLayer)	(None, 8, 8, 3)	0	
max_pooling2d (MaxPooling2D)	(None, 8, 8, 32)	0	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 8, 8, 32)	896	input_2[0][0]
concatenate (Concatenate)	(None, 8, 8, 64)	0	max_pooling2d[0][0] conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 8, 8, 64)	36928	concatenate[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 4, 4, 64)	0	conv2d_2[0][0]
flatten (Flatten)	(None, 1024)	0	max_pooling2d_1[0][0]
dense (Dense)	(None, 128)	131200	flatten[0][0]
dense_1 (Dense)	(None, 3)	387	dense[0][0]
=====			

Total params: 170,307
Trainable params: 170,307
Non-trainable params: 0

'Y' shape architecture

training

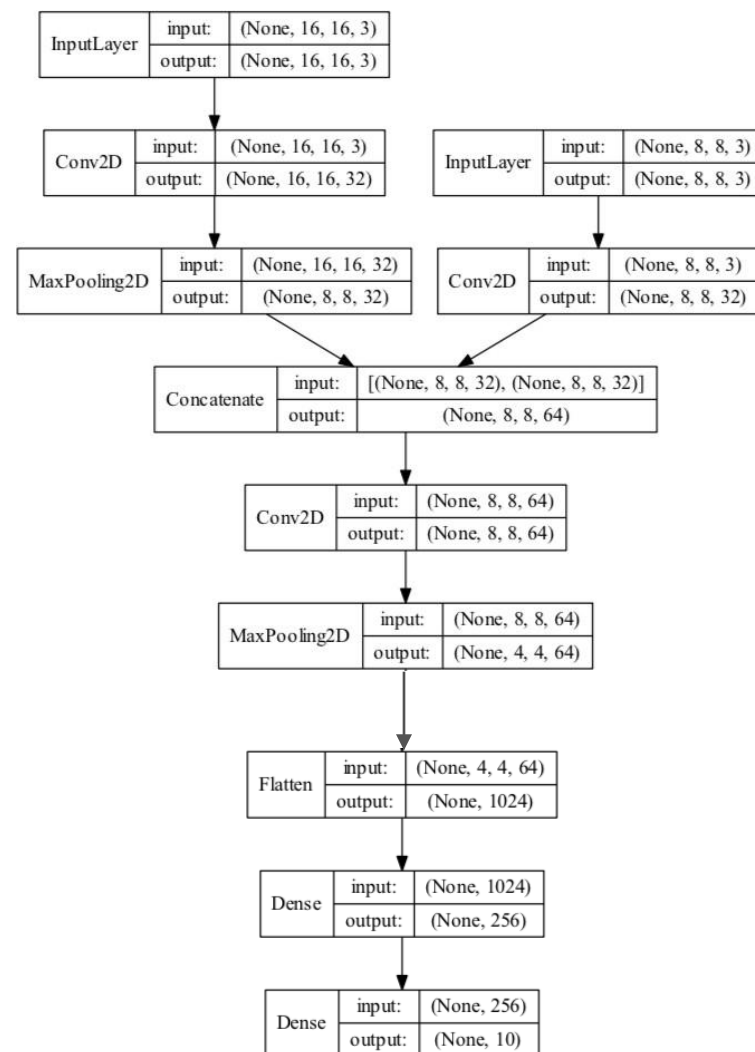
- Assume LDat and RDat are the training input, TLbl is the label for the training
- Furthermore, vLDat and vRDat is the validation input, and vLbl is the label
- The training is done by
 - > `model.fit([LDat,RDat],
TLbl,
validation_data=([vLDat,vRDat], vLbl),
epochs=100,
batch_size=128,
shuffle=True,
callbacks=callbacks_list)`

'Y' shape architecture

Getting the model plot

- To plot model, it requires pydot and graphviz

- On Mac, there is a need for additional installation for graphviz



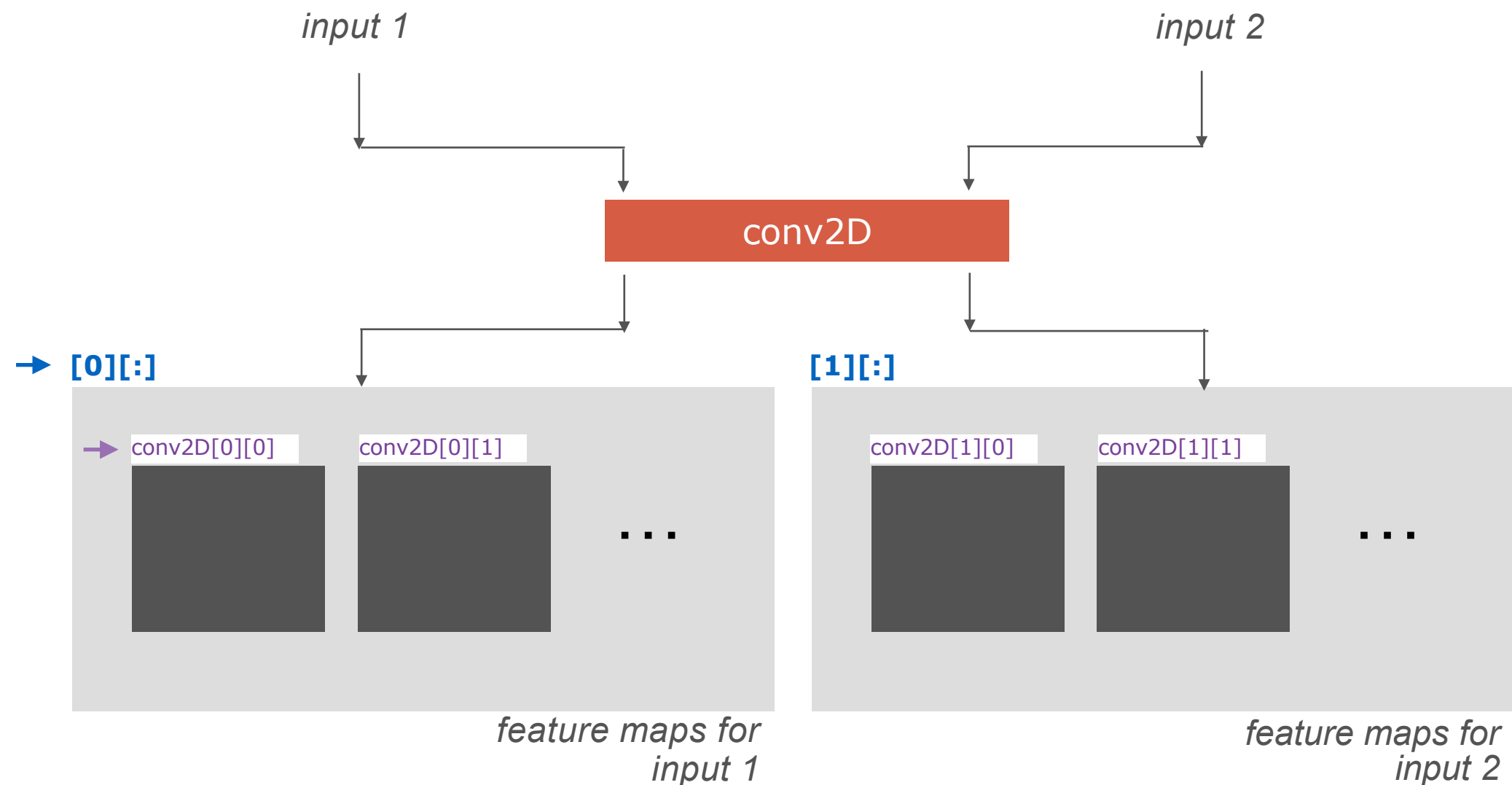
```
> from tensorflow.keras.utils import plot_model
```

```
> plot_model(modelDual,  
             to_file='Dual_model.pdf',  
             show_shapes=True,  
             show_layer_names=False,  
             rankdir='TB')
```

Shared layer

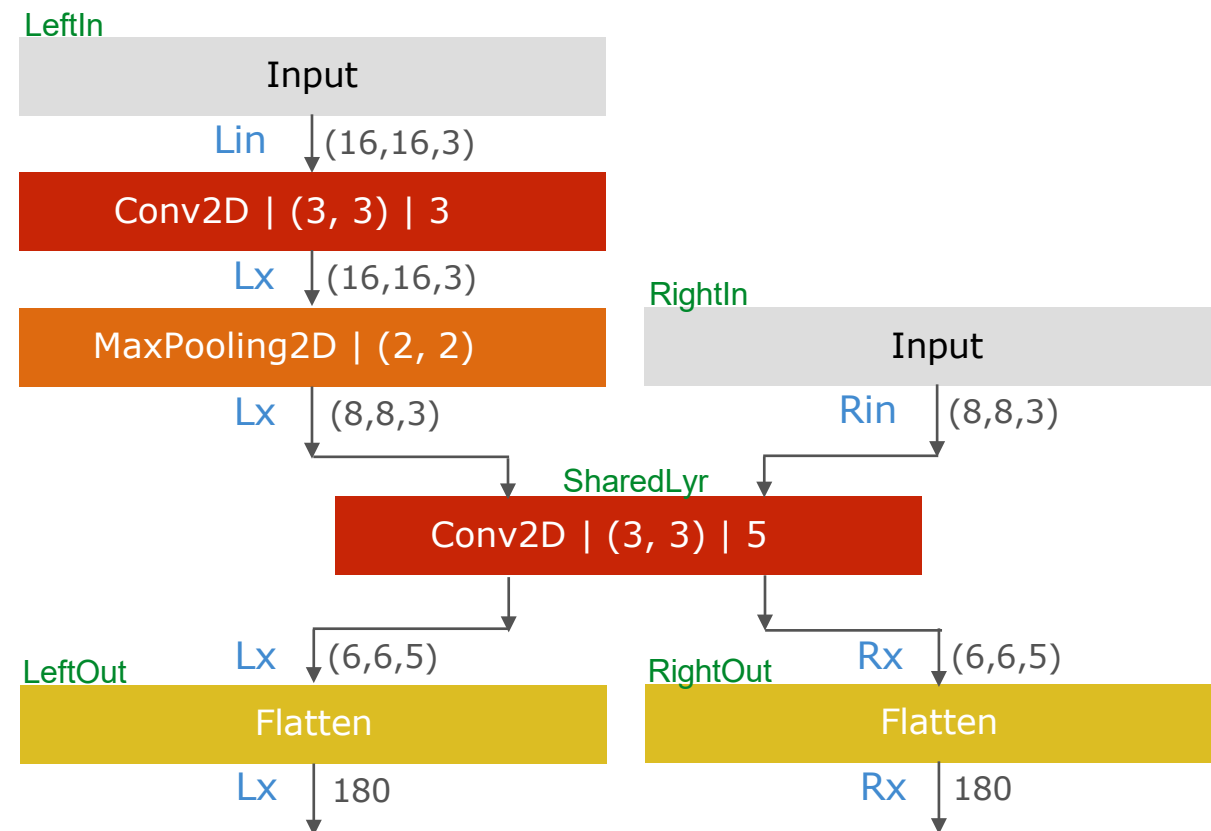
Applying weights to two input

- A single set of Conv2D weights is applied independently to two input streams



Shared layer

how to create?



Note: Shared layers do **not** require multiple outputs. We are showing multiple outputs here to demonstrate how to train a network with two separate predictions.

Shared layer

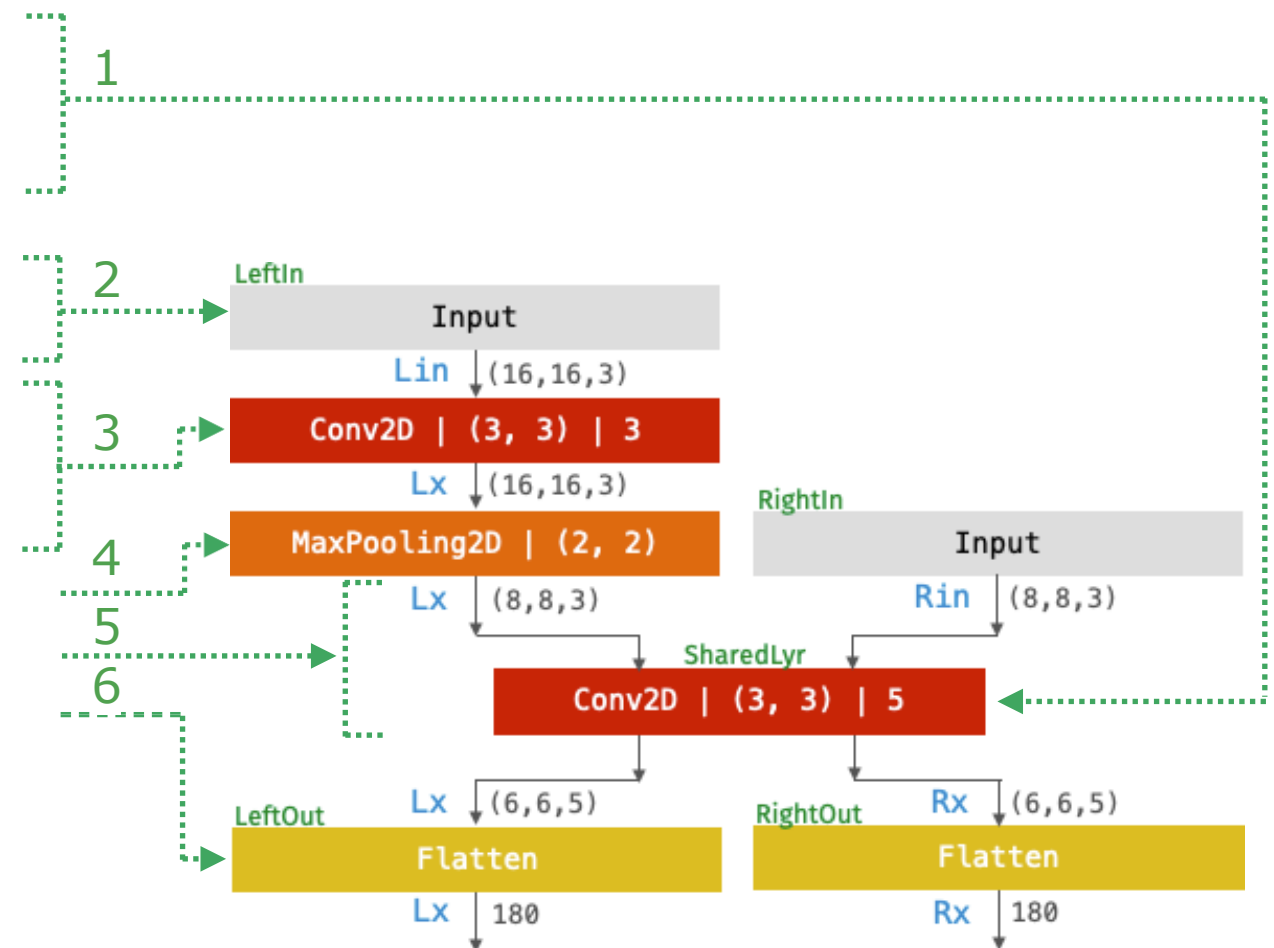
the code

```
> def createSharedModel():
    shared = Conv2D(5,(3,3),
                    activation='relu',
                    name='SharedLyr')

    Lin = Input(shape=(16,16,3),
                 name='LeftIn')
    Lx = Conv2D(3,(3,3),
               padding='same',
               activation='relu')(Lin)
    Lx = MaxPooling2D(pool_size=(2,2))(Lx)
    Lx = shared(Lx)
    Lx = Flatten(name='LeftOut')(Lx)

    Rin = Input(shape=(8,8,3),
                 name='RightIn')
    Rx = shared(Rin)
    Rx = Flatten(name='RightOut')(Rx)

    model = Model(inputs=[Lin,Rin],outputs=[Lx,Rx])
    ...
```



Note: Shared layers do **not** require multiple outputs. We are showing multiple outputs here to demonstrate how to train a network with two separate predictions.

Shared layer

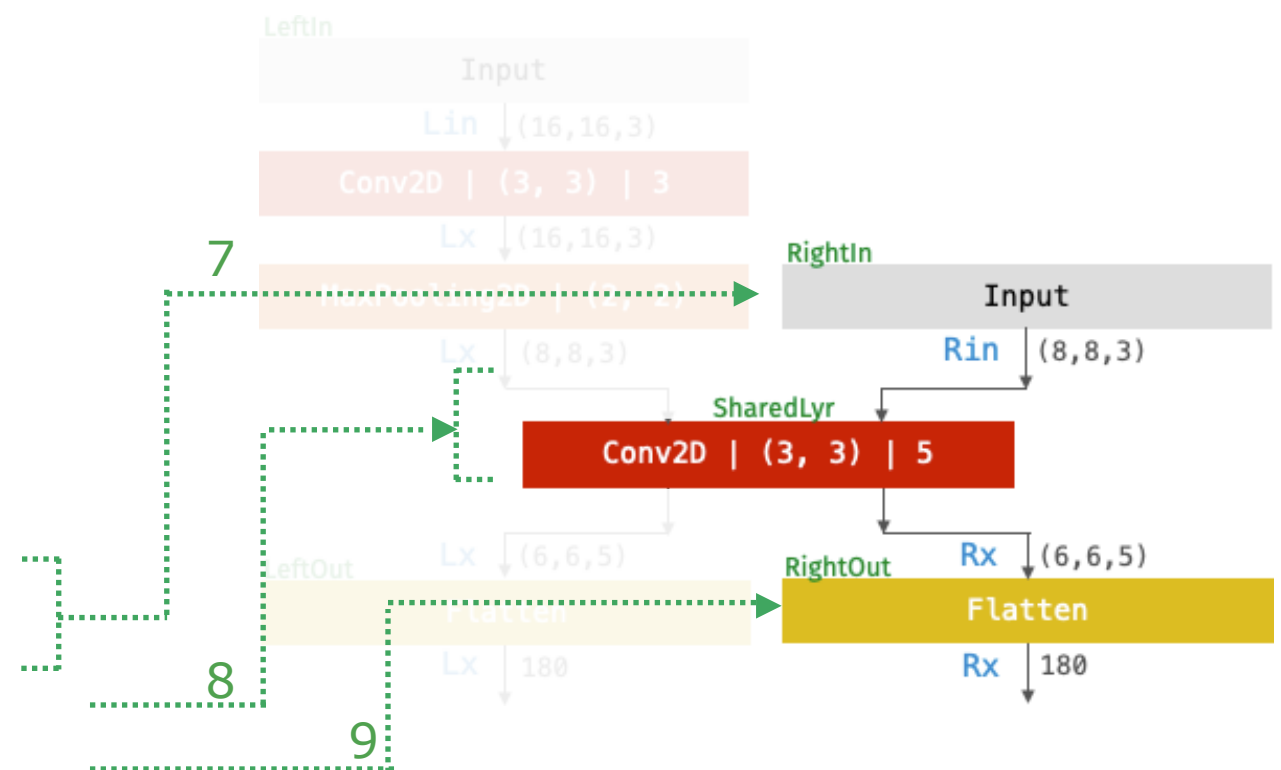
the code

```
> def createSharedModel():
    shared = Conv2D(5,(3,3),
                    activation='relu',
                    name='SharedLyr')

    Lin = Input(shape=(16,16,3),
                 name='LeftIn')
    Lx = Conv2D(3,(3,3),
               padding='same',
               activation='relu')(Lin)
    Lx = MaxPooling2D(pool_size=(2,2))(Lx)
    Lx = shared(Lx)
    Lx = Flatten(name='LeftOut')(Lx)

    Rin = Input(shape=(8,8,3),
                 name='RightIn')
    Rx = shared(Rin)
    Rx = Flatten(name='RightOut')(Rx)

    model = Model(inputs=[Lin,Rin],outputs=[Lx,Rx])
    ...
```



Note: Shared layers do **not** require multiple outputs. We are showing multiple outputs here to demonstrate how to train a network with two separate predictions.

Shared layer

the code

- For model with more than one output, pass in a dictionary that specifies the loss function for each output

```
> def createSharedModel():
    shared = Conv2D(5,(3,3),activation='relu',name='SharedLyr')

    Lin = Input(shape=(16,16,3),name='LeftIn')
    Lx = Conv2D(3,(3,3),padding='same',activation='relu')(Lin)
    Lx = MaxPooling2D(pool_size=(2,2))(Lx)
    Lx = shared(Lx)
    Lx = Flatten(name='LeftOut')(Lx)

    Rin = Input(shape=(8,8,3),name='RightIn')
    Rx = shared(Rx)
    Rx = Flatten(name='RightOut')(Rx)

    model = Model(inputs=[Lin,Rin],outputs=[Lx,Rx])
    model.compile(loss={'LeftOut':'categorical_crossentropy',
                        'RightOut':'mean_squared_error'},
                  optimizer='rmsprop', metrics=['accuracy'])

    return model
```



Note: Shared layers do **not** require multiple outputs. We are showing multiple outputs here to demonstrate how to train a network with two separate predictions.

Keras

Merge layer

- Other possible layers to fuse multiple flows of tensors in Keras

`tensorflow.keras.layers.add`

`tensorflow.keras.layers.subtract`

`tensorflow.keras.layers.multiply`

`tensorflow.keras.layers.average`

`tensorflow.keras.layers.maximum`

`tensorflow.keras.layers.minimum`

- For these element-wise layers, the inputs must have the same shape
- Good enough for most use cases. For custom math, you can use a **Lambda layer**

Group discussion

Suggest a use case / scenario / application where multiple inputs or multiple outputs are required for a deep neural network

Exercise

Suggest a use case / scenario / application where multiple inputs or multiple outputs are required for a deep neural network

- Make your proposal by answering the below questions:
- What is the problem to be solved? Why is the problem worth solving?
- What are the input(s) and output(s)?
Provide a few examples on the input and output (approximation is also OK)
- How do you plan to collect the data for your input and output, and what would you do in your data collection to assure robustness of your trained model
- How much data do you think is needed to train your model well?

Short exercise

Build the model using functional APIs

Short exercise

DualTw model

